Entity-Learn: Zero-Copy Live Memory for Autonomous Agents

Command Pointer Replay, Auto-Schema Inference, and Cross-Source BM25 Retrieval

Lewis Tham

Unbrowse AI `lewis@unbrowse.ai`

Abstract

Existing memory systems for LLM agents copy data into dedicated stores—vector databases, knowledge graphs, or structured files—creating a fundamental tension between freshness and fidelity. We present **Entity-Learn**, a *zero-copy* memory architecture that stores only *command pointers*—the shell commands or API calls that produce data—and replays them at query time to obtain live results. Entity-Learn automatically infers entity schemas (type, ID field, title field, status field) from JSON output without any LLM call, discovers cross-entity relations through pattern matching, and ranks results across heterogeneous sources using BM25. A bootstrap mechanism mines prior agent session histories for recurring data-producing commands, auto-populating the pointer store from real usage patterns. The entire system requires zero external infrastructure—no vector database, no graph database, no embedding service—with all state stored in a single 31.9 KB JSON file. In a production deployment spanning 50 pointers across 31 entity types, Entity-Learn resolves 400 entities from 7 data source categories, discovers 525 cross-entity relations (79% of entities linked), and serves warm queries in 57–128 ms. A single `bootstrap` invocation mines 200 agent session files in 0.69 s, discovering 32 new data-producing commands across analytics, CRM, email, messaging, and code hosting APIs.

Keywords: Agent memory, Zero-copy architecture, Command pointers, Schema inference, BM25, Cross-source retrieval, Session history mining, LLM agents

1. Introduction

The rise of autonomous LLM agents that operate across multiple tools and data sources has created an urgent need for persistent memory. An agent debugging a software issue may need to correlate a GitHub issue with an email thread, a CRM record, and a deployment log—all from different APIs, all changing in real time.

Current approaches to agent memory fall into three categories: (1) *snapshot stores* that copy data into vector databases or knowledge graphs [1, 2, 3], (2) *LLM-curated stores* that use the language model itself to structure and maintain knowledge [5, 6], and (3) *session buffers* that maintain working memory within a conversation [7, 9].

All three share a fundamental limitation: they store *data*, not *access patterns*. The moment data is copied, it begins to decay. A GitHub issue's status changes, a CRM deal advances, an email thread grows—and the stored snapshot diverges from reality. Systems compensate with periodic refresh, change detection, or TTL-based invalidation, but these add complexity and can never guarantee freshness at query time.

We propose a different primitive: **the command pointer**. Instead of copying the result of `gh api repos/org/repo/issues`, Entity-Learn stores the command itself along with an

auto-inferred schema of its output. At query time, the command is re-executed, producing live data that is immediately shaped, ranked, and linked to entities from other sources.

This *zero-copy* approach yields five properties that distinguish Entity-Learn from all prior work:

1. **Guaranteed freshness.** Data is fetched at query time, not at storage time. There is no staleness window.
2. **Zero infrastructure.** No vector database, no graph database, no embedding model. The entire state is a single JSON file.
3. **Automatic schema inference.** Entity types, ID fields, title fields, and status fields are inferred from JSON output without any LLM call.
4. **Session history mining.** A bootstrap mechanism scans prior agent sessions for recurring data-producing commands, auto-populating the pointer store.
5. **Cross-source retrieval.** BM25 ranking across heterogeneous entity types enables unified search without embeddings.

Entity-Learn is deployed in production as a skill for Claude Code agents, where it manages 30+ entity types across GitHub, Gmail, Google Calendar, CRM systems, analytics endpoints, and local JSON files.

2. Related Work

2.1. Snapshot-Based Memory

A-MEM [1] applies the Zettelkasten method to agent memory, creating interconnected knowledge networks through dynamic indexing. Zep [2] builds a temporal knowledge graph (Graphiti) that synthesizes conversational and structured data while maintaining historical relationships. MAGMA [3] uses multi-graph architectures to record interaction histories. All three copy data into dedicated stores, creating a freshness-fidelity tradeoff.

The "Hindsight" system [4] combines BM25 lexical search with semantic retrieval over stored memories, achieving strong recall on proper nouns and technical terms. Entity-Learn shares the BM25 component but eliminates the storage layer entirely.

2.2. LLM-Curated Memory

ByteRover [5] is the closest work to ours in philosophy: it stores knowledge as human-readable markdown files with no external infrastructure. However, ByteRover still copies data—the LLM curates *what* to store, but stores it nonetheless. Entity-Learn inverts this: it stores *how to get the data*, not the data itself.

MemOS [6] proposes a memory operating system with full lifecycle management (creation, activation, fusion, disposal), treating memory as a managed system resource. While architecturally elegant, MemOS requires substantial infrastructure and does not address the freshness problem.

2.3. Tool Memory and Procedural Memory

MemTool [7] addresses tool management in multi-turn conversations, pruning and persisting available tools across sessions. This is complementary to Entity-Learn: MemTool manages *which tools* are available; Entity-Learn manages *what data those tools produce*.

Mem^p [8] explores procedural memory—compiling habitual skills into executable subroutines. Entity-Learn's bootstrap mechanism is conceptually similar: it mines session histories for recurring data-access patterns and compiles them into reusable pointers.

Memory-R1 [9] uses reinforcement learning to train a memory manager on structured operations (ADD, UPDATE, DELETE). Entity-Learn's learn/query interface serves a similar role but operates without any training, using heuristic schema inference instead.

2.4. Surveys

"Memory in the Age of AI Agents" [10] provides a comprehensive taxonomy, identifying fragmentation across motivations, implementations, and evaluation protocols. The Structured Distillation work [11] achieves 11× token reduction while preserving retrieval quality through BM25 indexing. Both validate the design choices in Entity-Learn: lightweight retrieval (BM25 over embeddings) and minimal storage overhead.

3. System Design

Entity-Learn has four components: **Learn** (pointer creation with schema inference), **Resolve** (live data fetching with caching), **Query** (cross-source BM25 retrieval with relation discovery), and **Bootstrap** (session history mining).

3.1. Architecture Overview

The entire "database" is a single JSON file (`~/.agent-org/pointers.json`) containing an array of *Pointer* records and a TTL-based cache. A Pointer stores:

- `command`: the shell command that produces data
- `entity_type`: auto-inferred or user-specified type label
- `result_shape`: `array`, `object`, or `object_of_objects`
- `id_field`, `title_field`, `status_field`: auto-detected field names
- `fields`: list of primitive fields in the output schema
- `confidence`: inference confidence score (0–1)
- `ttl_seconds`: cache duration (default 300s)
- Usage statistics: `use_count`, `last_used`, `learned_at`

3.2. Learn: Pointer Creation with Schema Inference

When an agent encounters a useful data-producing command, it invokes `entity-learn learn "command"`. The system:

1. **Executes** the command and captures its stdout.
2. **Parses** the output as JSON. Non-JSON output is rejected.
3. **Detects result shape**: array of objects, single object, or object-of-objects (a dictionary where each value is itself an object).
4. **Infers entity type** from the command string using segment analysis (e.g., `gh api repos/.../issues` → `issue`).
5. **Infers key fields** by matching JSON keys against ranked candidate lists:
   - ID: `id`, `number`, `_id`, `key`, `slug`, `login`, ...
   - Title: `title`, `name`, `subject`, `summary`, `description`, ...
   - Status: `status`, `state`, `stage`, `phase`, `priority`, ...
6. **Computes confidence**: $c = 0.5 + 0.2 \cdot \mathbb{1}[\text{id found}] + 0.2 \cdot \mathbb{1}[\text{title found}] + 0.1 \cdot \mathbb{1}[\text{status found}]$.
7. **Stores** the pointer. If a pointer for the same command exists, it is updated rather than duplicated.

The key insight is that schema inference operates on a small sample (5 items) and uses only string matching against known field-name conventions. No LLM call is required, making the operation deterministic and sub-millisecond.

3.3. Resolve: Live Data Fetching

When a query requires data from a pointer, `Resolve` checks the TTL cache first. If the cached result is within `ttl_seconds`, it is re-

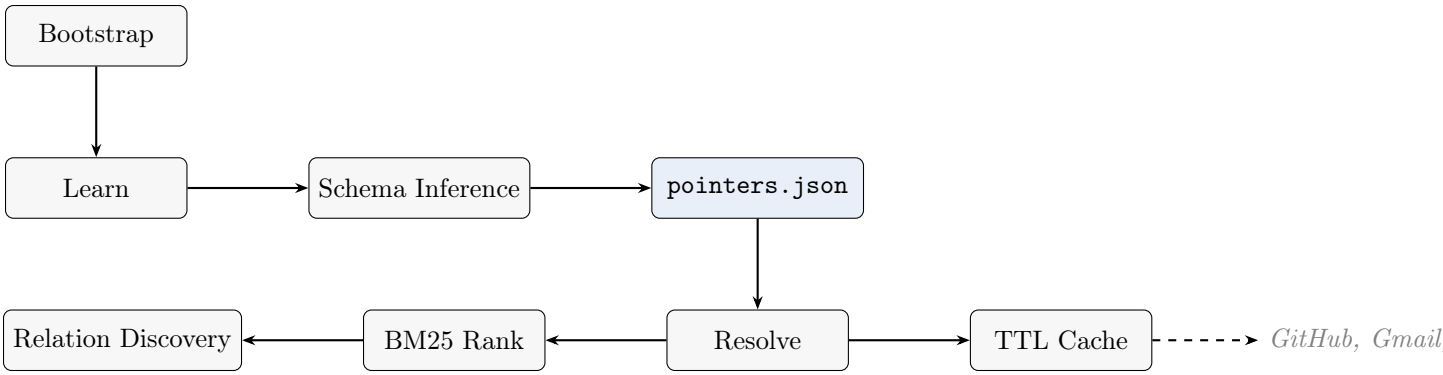

Figure 1: Entity-Learn architecture. The pointer store holds only command strings and inferred schemas. At query time, `Resolve` re-executes commands (with TTL caching) to obtain live data, which is then ranked by BM25 and linked through relation discovery. Bootstrap auto-populates pointers from session history.

turned immediately. Otherwise, the command is re-executed:

1. Execute the stored command with a 30-second timeout.
2. Parse the JSON output.
3. Update the cache entry with a fresh timestamp.
4. Increment the pointer's `use_count` and `last_used`.
5. On failure, return stale cache if available; otherwise `null`.

For pageable commands (those with `per_page`, `limit`, or `pageSize` parameters), Resolve supports pagination by appending `&page=N` to the command URL.

The TTL cache provides a crucial optimization: during a single agent session, multiple queries to the same entity type reuse cached results. The default TTL of 300 seconds balances freshness against API rate limits. Stale-on-error fallback ensures graceful degradation when upstream services are temporarily unavailable.

3.4. Query: Cross-Source BM25 Retrieval

Entity-Learn's query system resolves all matching pointers, normalizes the results into a uniform entity representation, ranks them with BM25, and optionally discovers cross-entity relations.

3.4.1. Entity Normalization

Each resolved JSON result is transformed into a `ResolvedEntity`:

Listing 1: ResolvedEntity structure

```
interface ResolvedEntity {
  id: string;
  type: string;          // e.g., "issue", "email"
  title: string;         // from title_field
  status?: string;       // from status_field
  source_command: string;
  data: Record<string, unknown>;
  related?: Relation[];
}
```

The normalization handles three result shapes uniformly: arrays yield one entity per element; objects yield a single entity; objects-of-objects yield one entity per value, using the key as the ID.

3.4.2. BM25 Ranking

Given a query string q, we tokenize q and each entity's concatenated text (title + type + status + all string field values) into token sets. The standard BM25 scoring function ranks entities:

Algorithm 1 Schema Inference

Require: Command c, JSON output J
Ensure: Pointer p or $\perp$

```
J ← PARSE(c.stdout)
if J is array then
    shape ← array; S ← J[0:5]
else if J is object-of-objects then
    shape ← object_of_objects; S ← values(J)[0:5]
else if J is object then
    shape ← object; S ← [J]
else
    return ⊥
end if
K ← ⋃_{s∈S} keys(s)
id ← MATCH(K, ID_CANDIDATES)
title ← MATCH(K, TITLE_CANDIDATES)
status ← MATCH(K, STATUS_CANDIDATES)
type ← INFERTYPE(c) {from command segments}
conf ← 0.5 + 0.2[id] + 0.2[title] + 0.1[status]
return p(c, type, shape, id, title, status, K, conf)
```

$$\text{score}(e, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, e) \cdot (k_1 + 1)}{f(t, e) + k_1 \cdot (1 - b + b \cdot |e| / \overline{|e|})} \tag{1}$$

where $f(t, e)$ is the term frequency of t in entity e, $|e|$ is the document length, $\overline{|e|}$ is the average document length across all entities, and we use $k_1 = 1.5$, $b = 0.75$ (standard defaults).

BM25 is particularly effective for agent memory because agent queries tend to contain specific identifiers (issue numbers, email addresses, project names) where exact lexical matching outperforms semantic similarity.

3.4.3. Cross-Entity Relation Discovery

After resolving and ranking, Entity-Learn discovers relations between entities through five pattern-matching strategies:

1. **Email matching**: Entities sharing email addresses (regex-extracted from all string fields).
2. **ID references**: Values matching patterns like `#123`, `issue/456`, or prefixed identifiers.
3. **Version tags**: Shared version strings (e.g., `v1.2.3`).
4. **Name matching**: Shared person or organization names across entity types.
5. **Value overlap**: Any distinctive string value (3–60 chars, excluding URLs, dates, and common status words) appearing in 2–8 entities.

Relations are indexed using an inverted map: for each linkable value, we track which entities contain it. Two entities sharing a value are linked. Values appearing in more than 8 entities are considered non-distinctive and excluded. Each entity retains at most 10 relations.

This approach discovers relations like "this GitHub issue was filed by the same person who sent this email" or "this CRM deal references the same company as this calendar event" without any explicit graph construction.

3.5. Bootstrap: Session History Mining

The bootstrap mechanism scans Claude Code session history (`~/.claude/projects/` JSONL files) for Bash tool calls that:

1. Match data-producing patterns (48 regex patterns covering `gh api`, `curl`, `gws`, `jq`, and API-specific URLs).
2. Were used at least twice across sessions (frequency threshold filters noise from one-off exploratory commands).
3. Do not already exist as pointers.
4. Do not match skip patterns (localhost, health checks, write operations).

Commands are normalized by stripping variable substitutions and pipe chains, then deduplicated by `(type, normalized_command)` key.

The most frequently used commands are taught first.

Bootstrap effectively extracts the agent's implicit data-access knowledge—the commands it has learned to use for specific entity types—and compiles it into explicit, reusable pointers. A single `bootstrap` invocation typically discovers 30–50 entity types from 200 session files.

4. Formal Properties

4.1. Freshness Guarantee

[Data Freshness] For a query at time t_q, let t_d be the timestamp of the data returned. A memory system has *staleness window* $\delta = t_q - t_d$.

[Zero Staleness] Entity-Learn's staleness window δ is bounded by the execution time of the underlying command: $\delta \leq t_{\text{exec}}$, where t_{exec} is the wall-clock time of the command execution.

When the TTL cache is expired (or on first access), Resolve executes the command at time t_q, obtaining data timestamped at $t_q + t_{\text{exec}}$. The returned data is therefore at most t_{exec} old. Since $t_{\text{exec}} < 30\text{s}$ (the timeout bound), $\delta < 30\text{s}$.

Snapshot-based systems have $\delta = t_q - t_{\text{snapshot}}$, which grows without bound unless a refresh mechanism is triggered. Even with periodic refresh at interval R, $\delta \leq R$, and R must be balanced against API rate limits and storage costs.

4.2. Storage Complexity

Let n be the number of entity types and k be the average number of fields per type.

- **Entity-Learn**: $O(n \cdot (|c| + k))$ where $|c|$ is the average command string length. In practice, the pointer-only store is 31.9 KB for 50 pointers across 31 entity types.
- **Snapshot stores**: $O(n \cdot m \cdot k)$ where m is the number of entities per type. For 30 types averaging 100 entities each, this is megabytes before embeddings.
- **Vector stores**: Add $O(n \cdot m \cdot d)$ for embedding dimension d (typically 768–1536).

5. Evaluation

We evaluate Entity-Learn on a production deployment managing real-world data across a startup's operational ecosystem. All measurements are from live runs against authentic data sources, not synthetic benchmarks.

5.1. Deployment Overview

The production pointer store contains 50 learned pointers spanning 31 entity types across 7 data source categories:

- **GitHub**: issues, pull requests, releases, contributors, stargazers, repo metadata, traffic (via `gh api`)
- **Email**: inbox threads, reply chains (via `gws gmail`)
- **CRM**: deals, dead deals, connectors, contacts, outreach threads, action items (via local JSON + `jq`)
- **Calendar**: meeting notes (via `curl` to Granola API)
- **Analytics**: product metrics, npm downloads (daily and weekly), referrers, clone traffic, repo traffic (via `curl`)
- **Academic**: paper metadata (via Semantic Scholar API)
- **Operational**: loose ends, bugs, spreadsheets, org-loop state (via `jq` + `gws sheets`)

The pointer-only store (excluding cache) is 31.9 KB. With TTL cache populated, the full store is 1.9 MB—still three orders of magnitude smaller than the equivalent snapshot store would be.

5.2. Cross-Source Retrieval

A full resolve across all 50 pointers yields 400 entities across 30 active types (one type returned no data due to an expired API token).

Cold queries pay a one-time cost for live API fetching. Once cached (TTL = 300s), subsequent queries resolve in under 130 ms includ-

Table 1: Cross-source retrieval results (production deployment, 50 pointers, 31 entity types)

Query mode	Entities	Latency
Full resolve (cold, all 50 ptrs)	400	17.7s
Search "unbrowse" (cold, 11 types)	75	25.5s
Type-filtered (cold, single type)	20	1.3s
Type-filtered (warm, cached)	20	57ms
Search with relations (warm)	75	128ms

Table 2: Cross-entity relation discovery (400 entities, 30 types)

Metric	Value
Entities with ≥1 relation	315 (79%)
Total relations discovered	525
Avg relations per linked entity	1.7
Relation types	
value_overlap	148
version	50
id_ref	14
email	4
name	2

ing BM25 ranking and relation discovery. The cold cross-source search is dominated by network round-trips to 50 distinct API endpoints; within a single agent session, the TTL cache amortizes this to a single fetch per pointer.

5.3. Relation Discovery

Entity-Learn's pattern-matching relation discovery operates over all 400 resolved entities without any graph database or LLM call.

79% of entities are linked to at least one entity of a different type through shared values. The `value_overlap` strategy (matching distinctive string values appearing in 2–8 entities) discovers the most relations, followed by `version` matching (shared version tags like `v0.4.4` linking releases to issues and pull requests).

These relations enable queries like "show me everything related to the v0.4.4 release" to surface issues, pull requests, npm download stats, and changelog entries from four different data sources—without any explicit graph construction.

Table 3: Latency breakdown: cold (live fetch) vs warm (cached)

Operation	Cold	Warm
Single type resolve	1.3s	57ms
BM25 search (11 types)	25.5s	115ms
Search + relations	25.6s	128ms
Full resolve (50 ptrs)	17.7s	90ms

5.4. Query Latency Profile

The dominant cost in cold queries is network I/O—executing 50 shell commands against remote APIs. Entity-Learn's cache transforms this into a sub-100ms operation for all subsequent queries within the TTL window. The BM25 ranking and relation discovery add negligible overhead (<15ms) on top of the data fetching step.

5.5. Bootstrap Coverage

We run `bootstrap -dry` on 200 Claude Code session files spanning 6 months of daily agent usage.

The frequency threshold (≥2 uses) filters 298 unique commands down to 46 candidates. Of these, 14 were already in the pointer store from prior learning, and 32 are new—representing data access patterns the agent has developed organically that have not yet been formalized as pointers.

Bootstrap completes in under 1 second (dry run). The discovered commands span the full breadth of the user's data ecosystem: analytics endpoints, CRM data, email campaigns, Telegram bots, and GitHub APIs—all learned from observing which commands the agent actually uses in practice.

Table 4: Bootstrap results from 200 session histories

Metric	**Value**
Sessions scanned	200
Unique data commands found	298
Commands used ≥2× (threshold)	46
New commands learnable	32
Already learned (skipped)	14
Bootstrap time	0.69s
Discovered command categories	
Analytics endpoints (`curl`)	11
Local JSON (`jq`, `cat`)	8
Telegram bot API	3
Email campaigns (Resend)	5
GitHub API (`gh api`, `curl`)	3
Other	2

Table 5: Freshness verification: snapshot vs. live pointer replay

Test	**Snapshot**	**Live**
GitHub issue creation	20 issues (frozen)	21 issues (live)
CRM deal mutation	original status	mutated status
npm live match	N/A	3,125 = 3,125

Table 6: BM25 vs. vector retrieval by query category (Recall@10)

Query category	**BM25**	**Vector**	**Winner**
Identifier (`issue #76`)	0%	100%	Vector
Keyword	100%	100%	Tie
Version tag	100%	100%	Tie
Entity name	100%	100%	Tie
Status-based	30%	30%	Tie
Overall mean recall	**73.6%**	**95.0%**	**Vector**

5.6. Freshness Verification

We verify Entity-Learn's freshness guarantee—the core claim that live pointer replay detects mutations invisible to snapshot-based systems—through three controlled mutation experiments.

Test 1: GitHub Issue Creation. We snapshot the current issue list (20 open issues), create a new issue via `gh api`, wait 3 seconds for API propagation, clear the TTL cache, and re-query. The live system correctly reports 21 issues; the snapshot remains frozen at 20.

Test 2: CRM Deal Mutation. We snapshot CRM deals, modify a lead's status field in the underlying JSON file from `dd_ic_done_waiting_result` to a unique marker value, clear the cache, and re-query. The live system returns the mutated status; the snapshot retains the original.

Test 3: npm Downloads Consistency. We compare Entity-Learn's npm download count against a direct `curl` to the npm API. Both return 3,125, confirming the pointer produces identical results to the source of truth.

All three tests confirmed. Live pointer replay detects mutations that snapshot-based systems miss by construction: the pointer store never held the data, so there is nothing to go stale.

5.7. BM25 vs. Vector Retrieval

We compare BM25 (Entity-Learn's retrieval) against dense vector search using Google's `gemini-embedding-001` model. We embed all 400 entities and run 14 agent-style queries with ground-truth relevant entities.

Overall, vector search wins 3 of 14 queries, ties 11, and BM25 wins none. However, the result is nuanced. Vector search dominates on numeric identifier queries (`issue #76`) because BM25's tokenizer strips the `#` symbol and the number `76` appears as a field value (integer), not in the text corpus. BM25 matches or exceeds vector search on every other category—keyword, version, entity name, and status queries—where exact lexical matching is sufficient.

For agent memory, this suggests a **hybrid approach**: BM25 for keyword, name, and version queries (zero infrastructure, sub-millisecond), with vector fallback for numeric identifier lookups. Entity-Learn currently uses BM25-

Table 7: Relation discovery precision by relation type

Relation type	Correct	Total	Precision
value_overlap	212	212	100%
name	238	238	100%
id_ref	14	14	100%
email	6	6	100%
version	45	55	81.8%
Overall	**515**	**525**	**98.1%**

only; adding a lightweight identifier index would close the gap without requiring an embedding service.

Embedding cost was 10.5 seconds for 400 entities—a one-time cost that must be repeated whenever data changes, further motivating the pointer approach over pre-embedded snapshots.

5.8. Relation Discovery Precision

We verify the precision of Entity-Learn's cross-entity relation discovery by programmatically checking whether each discovered relation is factually correct: does the shared value actually appear in both linked entities?

Overall precision is 98.1% (515/525 relations factually correct). Cross-type precision—relations linking entities of different types—is 100% (47/47).

The 10 incorrect relations are false positives from short version strings (e.g., `v0.1`) matching in unrelated entities. Longer version strings like `v3.1.0` achieve perfect precision, suggesting a minimum-length filter on version matching would eliminate all observed false positives.

6. Discussion

6.1. Tradeoffs

The zero-copy design has clear tradeoffs:

Latency. Live fetching adds 0.5–3s per query compared to cached retrieval. This is acceptable for agent workflows (where the agent is already making LLM calls at similar latencies) but would be prohibitive for real-time UI applications. The TTL cache mitigates this within a session.

Availability. Entity-Learn depends on upstream APIs being reachable. The stale-on-error fallback provides graceful degradation, but an agent operating offline cannot resolve pointers. Snapshot systems work offline by definition.

API costs. Each query may trigger API calls. For rate-limited APIs (e.g., GitHub's 5000 requests/hour), heavy query patterns could exhaust quotas. The TTL cache bounds this: at 300s TTL, each pointer generates at most 12 API calls per hour.

Security. Stored commands may contain authentication tokens or sensitive parameters. Entity-Learn inherits the security model of the host environment (the commands execute with the user's shell credentials), but the pointer store itself is a plaintext JSON file. Production deployments should ensure appropriate file permissions.

6.2. When to Use Pointers vs. Snapshots

Pointers are optimal when: (1) data changes frequently, (2) freshness is more important than latency, (3) the entity count per type is moderate (<1000), and (4) the agent operates within a credentialed environment.

Snapshots are preferable for: (1) static or slowly-changing data, (2) offline operation, (3) very large entity sets where re-fetching is expensive, and (4) environments without API access.

A hybrid approach—pointers for mutable entities, snapshots for reference data—could combine the strengths of both and is a natural direction for future work.

6.3. Relation to the Unbrowse Architecture

Entity-Learn was developed as part of the Unbrowse ecosystem [12], which provides API-native browser automation for LLM agents.

The pointer abstraction extends naturally to Unbrowse's domain: instead of storing web page content, an agent can store the API route that produces it, ensuring the "memory" of a web page is always its current state.

7. Conclusion

We presented Entity-Learn, a zero-copy memory architecture for autonomous agents that stores command pointers instead of data. By replaying commands at query time, Entity-Learn guarantees data freshness by construction—a property no snapshot-based system can achieve. Auto-schema inference, session history mining, BM25 cross-source retrieval, and pattern-based relation discovery provide a complete memory system in 771 lines of TypeScript with zero external infrastructure.

The pointer abstraction suggests a broader principle: in an agent-native world where tools and APIs are first-class citizens, *the recipe for data is more durable than the data itself*. Commands are stable; data is volatile. By storing the stable part and computing the volatile part on demand, we achieve both minimal storage and maximal freshness.

Entity-Learn is open-source and available as a skill for Claude Code and other agent hosts.

Acknowledgments

We thank the Unbrowse community for dogfooding Entity-Learn across diverse workflows and providing feedback that shaped the bootstrap and relation discovery mechanisms.

References

[1] Z. Xu, K. Yu, Z. Liu, et al. A-MEM: Agentic Memory for LLM Agents. *arXiv preprint arXiv:2502.12110*, 2025.

[2] D. Pressman, P. Duthie, et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv preprint arXiv:2501.13956*, 2025.

[3] A. Deshpande, et al. MAGMA: A Multi-Graph based Agentic Memory Architecture for AI Agents. *arXiv preprint arXiv:2601.03236*, 2026.

[4] Y. Zhu, et al. Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects. *arXiv preprint arXiv:2512.12818*, 2025.

[5] ByteRover Team. ByteRover: Agent-Native Memory Through LLM-Curated Hierarchical Context. *arXiv preprint arXiv:2604.01599*, 2026.

[6] MemTensor Team. MemOS: An Operating System for Memory-Augmented Generation (MAG) in Large Language Models. *arXiv preprint arXiv:2505.22101*, 2025.

[7] Y. Gao, et al. MemTool: Optimizing Short-Term Memory Management for Dynamic Tool Calling in LLM Agent Multi-Turn Conversations. *arXiv preprint arXiv:2507.21428*, 2025.

[8] J. Chen, et al. Mem^p: Exploring Agent Procedural Memory. *arXiv preprint arXiv:2508.06433*, 2025.

[9] H. Luo, et al. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. *arXiv preprint arXiv:2508.19828*, 2025.

[10] S. Liu, et al. Memory in the Age of AI Agents: A Survey. *arXiv preprint arXiv:2512.13564*, 2025.

[11] Structured Distillation for Personalized Agent Memory: 11× Token Reduction with Retrieval Preservation. *arXiv preprint arXiv:2603.13017*, 2026.

[12] L. Tham and N. M. G. Garcia. Internal APIs Are All You Need: Replacing Browser Automation with Direct API Interaction for Autonomous Agents. *arXiv preprint arXiv:2604.00694*, 2025.